

PAPER

1. ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation

1. Introduzione e motivazioni

Il rapido sviluppo di tecnologie come i dispositivi indossabili per la realtà aumentata, i sistemi di automazione domestica e i veicoli a guida autonoma ha aumentato la richiesta di algoritmi di **segmentazione semantica in tempo reale** su dispositivi mobili a basso consumo energetico. Questo compito consiste nell'etichettare ogni pixel di un'immagine, fornendo una comprensione completa della scena.

Le reti convoluzionali (CNN) applicate alla segmentazione, producono mappe grossolane e spesso necessitano di passaggi di post-elaborazione (come CRF o segmentazione basata sul colore) per affinare i risultati.

Sono state proposte architetture come **SegNet** e le **Fully Convolutional Networks**, derivate da VGG16. Questi modelli hanno un grande numero di parametri e tempi di inferenza troppo elevati, che li rendono inutilizzabili per applicazioni mobili. Altri approcci combinano CNN con RNN o CRF: migliorano l'accuratezza ma aumentano notevolmente i costi computazionali. In risposta a questi limiti, gli autori introducono **ENet**, un'architettura progettata appositamente per la segmentazione semantica in tempo reale, *con l'obiettivo di massimizzare l'efficienza mantenendo buoni livelli di accuratezza*, ed evitando del tutto la dipendenza da post-processing esterni.

2. Architettura di ENet

ENet adotta una struttura **encoder-decoder**:

- **Encoder** grande: comprime l'informazione riducendo la risoluzione
- **Decoder** piccolo: si limita a ricostruire la mappa segmentata, refinendo i dettagli, senza replicare la complessità dell'encoder.

2.1 Struttura generale

- Ispirata a **ResNet**, con un **ramo principale** e rami secondari che si staccano ed eseguono convoluzioni, per poi ricongiungersi con una **somma elemento per elemento**.
- Ogni **bottleneck** è composto da tre operazioni:
 1. **Proiezione 1×1** per ridurre la dimensionalità.
 2. **Convoluzione principale** (regolare, dilatata, deconvoluzione o asimmetrica).
 3. **Espansione 1×1** per ripristinare i canali.
- Tra queste operazioni vengono inserite **Batch Normalization** e **PReLU**.
- Se il bottleneck esegue un **downsampling**, al ramo principale si aggiunge anche un **max pooling**.

2.2 Moduli principali

- **Blocco iniziale**: combina in parallelo un pooling 2×2 e una convoluzione con stride 2; i risultati vengono concatenati. Questa scelta riduce subito la dimensione spaziale senza perdita significativa di informazione.
- **Convoluzioni principali**:
 - **Regolari 3×3**,
 - **Dilatate**, per aumentare il campo recettivo,
 - **Full** (deconvoluzione/fractionally strided),
 - **Asimmetriche 5×1 + 1×5**, che riducono i costi mantenendo la capacità espressiva.
- **Decoder**: usa upsampling tramite **max unpooling**. L'ultimo livello è una **full convolution** che genera direttamente la mappa delle classi.

3. Principi di design

3.1 Downsampling precoce

Elaborare immagini ad alta risoluzione è costoso. Per questo ENet riduce subito la dimensione spaziale e lavora con poche feature map.

3.2 Cambiamenti dimensionali “information-preserving”

L’approccio VGG (pooling → convoluzione che espande) introduce un collo di bottiglia, mentre conv → pooling è troppo costoso. ENet adotta una soluzione parallela: pooling e conv stride 2 insieme, con concatenazione dei risultati.

3.3 Fattorizzazione dei filtri

ENet adotta conv asimmetriche con $n=5$, equivalenti in costo a una conv 3×3 , ma con maggiore campo recettivo e varietà funzionale.

3.4 Convoluzioni dilatate

ENet usa convoluzioni dilatate negli stadi a risoluzione ridotta, intervallandole con moduli regolari o asimmetrici. Questo ha migliorato la IoU su Cityscapes.

3.5 Non-linearità

Gli autori hanno osservato che usare ReLU + BN prima delle conv peggiora i risultati. Rimuovere molte ReLU nelle prime fasi ha migliorato l’accuratezza. Per questo ENet usa **PReLU** in tutti i layer.

3.6 Regularizzazione

I dataset di segmentazione pixel-wise hanno poche migliaia di immagini: le reti rischiano di overfittare. **Spatial Dropout** si è dimostrata ancora superiore. Viene applicata alla fine dei rami convoluzionali, prima della somma.

4. Valutazione su dataset

ENet è stato testato su tre dataset per verificare accuratezza e tempo reale.

- **Cityscapes**: 5000 immagini urbane annotate finemente, 19 classi. ENet supera SegNet in class IoU, category IoU e mIoU, ed è il modello più veloce del benchmark.
- **CamVid**: 367 immagini di training e 233 di test, 11 classi. ENet eccelle soprattutto nelle classi piccole e difficili da apprendere, superando gli altri modelli in sei categorie.
- **SUN RGB-D**: 5285 immagini di training e 5050 di test, 37 classi indoor. ENet ottiene un’accuratezza media per classe comparabile a SegNet, pur essendo inferiore in IoU e accuratezza media globale. Tuttavia, la velocità è molto più alta: quasi 20× su piattaforme embedded.

5. Analisi delle prestazioni

5.1 Setup sperimentale

Gli esperimenti sono stati condotti su **Torch7** con backend **cuDNN**. Come baseline è stato scelto **SegNet**. Le metriche usate sono accuratezza media per classe e IoU.

5.2 Risultati di velocità

Su **NVIDIA Titan X**, ENet raggiunge oltre 100 fps a risoluzioni intermedie (640×360), mentre SegNet si ferma a circa 15 fps.

Su **NVIDIA TX1** (embedded), ENet mantiene oltre 10 fps su input 640×360 , adatti a parsing stradale reale. SegNet, invece, resta ben sotto 2 fps.

5.3 Efficienza hardware

Con input 640×360 , ENet richiede circa **3.8 GFLOPs**, contro i 286 di SegNet. Ha 0.37 milioni di parametri contro 29.4 milioni e un modello da 0.7 MB (fp16), che può risiedere interamente in memoria on-chip.

6. Limitazioni software

La **fattorizzazione dei filtri**, che riduce FLOPs e parametri, comporta un aumento delle chiamate ai kernel GPU, ciascuna più piccola.

Il problema è particolarmente evidente con le **PReLU**, operazioni semplici ma che consumano oltre un quarto del tempo di inferenza. Qui il costo non è nel calcolo, ma nella gestione della memoria e nel lancio dei kernel.

Gli autori suggeriscono la **fusione dei kernel** nelle librerie (es. cuDNN): unificare convoluzioni e non-linearità in un'unica chiamata, o raggruppare più conv piccole insieme. Questo miglioramento software potrebbe aumentare ulteriormente la velocità di ENet.

2. ICNet for Real-Time Semantic Segmentation on High-Resolution Images

1. Introduzione

Le CNN moderne hanno migliorato molto le prestazioni, ma l'accuratezza tipicamente cresce insieme a profondità, larghezza e quindi costo computazionale. Nei compiti pixel-wise, ottenere più accuratezza tende a richiedere molte più operazioni, specialmente su immagini ad alta risoluzione.

La ricerca sulla velocità è rimasta indietro rispetto a quella sull'accuratezza.

Obiettivo: costruire un sistema veramente veloce con accuratezza "decente". ICNet è tra i pochissimi approcci real-time che mantengono una buona qualità.

Contributi principali:

- nuova ICNet real-time che sfrutta efficacemente semantica da bassa risoluzione e dettagli da alta risoluzione;
- CFF + guida a cascata per recupero e raffinamento progressivi a basso costo;
- ~5× di speed-up e ~5× di riduzione memoria; ~30 fps a 1024×2048 con qualità elevata su Cityscapes, CamVid e COCO-Stuff.

3. Image Cascade Network

3.1 Analisi della velocità

Il costo di una rete di segmentazione cresce quadraticamente con la risoluzione delle feature map e aumenta anche quando si raddoppiano canali d'ingresso/uscita in blocchi profondi. Nelle architetture standard, gli stadi finali possono costare molto più dei precedenti pur avendo la stessa risoluzione intermedia, perché usano molti più canali. In breve: risoluzione e larghezza sono i driver principali del tempo d'inferenza.

3.2 Architettura

Le strategie "semplici" (ridurre risoluzione in ingresso, restringere le feature, potare i filtri) accelerano, ma degradano troppo la qualità; processare direttamente la full-res con backbone pesanti è impraticabile.

ICNet evita l'alternativa secca:

- usa tre rami con ingressi a cascata (bassa, media, alta risoluzione);
- il ramo basso (1/4 della risoluzione) applica un backbone di segmentazione e produce una mappa grossolana a basso costo;
- i rami medio e alto, leggeri, recuperano confini e piccoli oggetti tramite CFF
- parte dei pesi/computazioni è condivisa tra rami, riducendo ulteriormente il costo. In pratica, l'estrazione semantica a bassa risoluzione, la fusione e il raffinamento finale si mantengono entro decine di millisecondi, con memoria contenuta.

3.3 Cascade Feature Fusion (CFF)

La CFF fonde le feature di bassa e media risoluzione così: si allinea spazialmente la mappa a bassa risoluzione con un upsampling efficiente e la si rifinisce con una convoluzione dilatata leggera; si proietta la mappa di media risoluzione al medesimo numero di canali; si normalizzano entrambe, si sommano e si passa per ReLU; sul ramo a bassa risoluzione si aggiunge una perdita ausiliaria. Questa procedura ottiene stesso campo recettivo effettivo di una deconvoluzione “grande”, ma con meno costo.

3.4 Guida a cascata delle etichette

Ogni ramo (basso, medio, alto) è supervisionato con etichette ground-truth ridotte (es. 1/16, 1/8, 1/4), con pesi di loss scelti per stabilizzare l’ottimizzazione. In test, le supervisioni a bassa/medio risoluzione sono rimosse: resta solo il ramo ad alta risoluzione come uscita finale. Questo rende il training più stabile e impedisce che un ramo domini la predizione.

5. Valutazione sperimentale

Dataset:

- Cityscapes (1024×2048; 5.000 immagini; 19 classi usate), sfida critica per il real-time.
- CamVid (720×960; 11 classi).
- COCO-Stuff (fino a 640×640; 182 classi, 91 “thing” + 91 “stuff”)

Strategie intuitive e limiti (Cityscapes)

- Ridurre la risoluzione d’ingresso accelera ma produce mappe coarse: si perdono piccoli oggetti e confini; con risoluzioni intermedie migliora, ma resta lento per il real-time.
- Ridurre drasticamente la risoluzione delle feature accelera ma perde dettagli stratificati nei layer bassi; anche con forti riduzioni, il tempo resta elevato.
- Potare i filtri (compressione L1) non basta: tempi ancora troppo lunghi e mIoU troppo bassa.

ICNet (ablation qualitativa)

- Solo ramo basso: velocissimo ma crollo della qualità.
- + ramo medio: grande recupero di dettagli, costo aggiuntivo modesto.
- + ramo alto: qualità vicina al baseline “pesante”, con tempo molto inferiore e memoria significativamente ridotta.

Analisi qualitativa delle predizioni

- Il ramo basso cattura già molte regioni semantiche, ma resta grossolano e perde regioni piccole (es. pali, semafori).
- Con il ramo medio si recuperano molte regioni piccole; permangono però alcuni confini sfumati e persone lontane mancanti.
- Con tutti i rami si rifiniscono i dettagli: confini più netti e piccoli oggetti meglio riconosciuti.

3. On Calibration of Modern Neural Network

1. Introduzione

Le reti neurali profonde hanno raggiunto livelli di accuratezza molto elevati. In questi contesti, tuttavia, non basta che il modello sia accurato: è altrettanto importante che sappia riconoscere quando le sue predizioni sono incerte o potenzialmente sbagliate.

Gli autori osservano che le reti storiche risultavano ben calibrate, mentre le architetture moderne mostrano spesso un’eccessiva sicurezza: la confidenza media delle predizioni è superiore all’accuratezza effettiva.

2. Definizioni

Si considera il problema della classificazione multiclasse supervisionata. Un modello è calibrato se, tra le istanze a cui assegna una probabilità $\sim p$, circa una frazione p risulta corretta.

Strumenti principali:

- **Diagrammi di affidabilità:** mostrano l'accuratezza media per fasce di confidenza; la distanza dalla diagonale segnala miscalibrazione.
- **ECE (Expected Calibration Error):** un singolo valore che riassume lo scarto medio tra accuratezza e confidenza, calcolato su più intervalli di confidenza ("bin").
- **MCE (Maximum Calibration Error):** lo scarto massimo osservato tra accuratezza e confidenza nei vari bin.
- **NLL (Negative Log-Likelihood):** misura standard della qualità probabilistica;

3. Osservare la miscalibrazione

- **Capacità del modello** → Aumentare profondità e larghezza migliora l'accuratezza, ma tende a peggiorare la calibrazione.
- **Batch Normalization** → Migliora l'ottimizzazione e spesso anche l'accuratezza, ma porta frequentemente a reti più mal calibrate rispetto a quelle senza, a parità di condizioni.
- **Weight decay** → La riduzione della regolarizzazione L2 peggiora la calibrazione. Al contrario, un weight decay più alto migliora la calibrazione.
- **Disaccoppiamento NLL vs accuratezza** → Dopo che l'errore di training si è quasi azzerato, il modello continua a ridurre il NLL aumentando artificialmente le confidenze. Si ottiene così una migliore accuratezza ma una peggiore calibrazione (overfitting "probabilistico").

4. Metodi di calibrazione (post-processing)

Tutti richiedono un validation set separato e agiscono senza riaddestrare la rete.

Per modelli binari (estendibili al multiclasse):

- **Histogram Binning:** suddivide le predizioni in intervalli di confidenza e assegna a ciascun intervallo la frequenza osservata di correttezza.
- **Isotonic Regression:** apprende una funzione non decrescente che mappa le confidenze in valori calibrati.
- **BBQ (Bayesian Binning into Quantiles):** media bayesiana di più schemi di binning.
- **Platt Scaling:** applica una regressione logistica sui logit per correggere le confidenze.

Estensioni al multiclasse:

- **Vector/Matrix Scaling:** generalizzazioni del Platt scaling applicate al vettore dei logit.
- **Temperature Scaling:** introduce un solo parametro T che ammorbidisce la softmax. Non modifica le classi predette (quindi non altera l'accuratezza), ma solo le confidenze.

Costi computazionali e praticità:

- Il **temperature scaling** è il più rapido e il più facile da integrare.
- **Vector/matrix scaling** hanno costo crescente con il numero di classi.
- **Binning, isotonic e BBQ** sono più lenti e possono cambiare le predizioni.

4. BiSeNet: Bilateral Segmentation Network for Real-time Semantic Segmentation

1. Introduzione

La segmentazione semantica assegna etichette a livello di pixel ed è fondamentale in *augmented reality*, guida autonoma e videosorveglianza, dove è cruciale l'inferenza veloce. Tre strategie comuni per accelerare: (1) ridurre l'input (crop o *resize*), semplice ma penalizzante per i dettagli (specie ai bordi); (2) pruning di canali (soprattutto negli stadi iniziali), che indebolisce la capacità spaziale; (3) eliminare stadi (es. ENet rimuove l'ultimo *downsampling*),

ma così il *receptive field* non copre oggetti grandi. In tutti i casi la velocità si ottiene sacrificando l'accuratezza.

Una risposta frequente è la struttura a U (U-shape), che fonde feature gerarchiche del backbone per recuperare in parte la risoluzione. Tuttavia: (1) la U completa rallenta per il costo su feature ad alta risoluzione; (2) molte informazioni spaziali perse con *crop* o *pruning* non si recuperano facilmente.

Proposta: BiSeNet con due percorsi complementari.

2. Bilateral Segmentation Network (BiSeNet)

3.1 Spatial Path

Obiettivo: preservare la dimensione spaziale dell'input. Lo SP ha tre strati (conv stride 2 + BN + ReLU) producendo feature a 1/8 della risoluzione originale: abbastanza grandi da mantenere dettagli, con costo computazionale modesto.

3.2 Context Path

Obiettivo: fornire sufficiente *receptive field* in modo efficiente. Il CP usa un backbone leggero che *downsample* velocemente e aggiunge un global average pooling in coda per catturare contesto globale. Nel backbone adottiamo una U-shape incompleta (8s) che fonde gli ultimi due stadi, limitando il costo.

Attention Refinement Module (ARM) → Per ciascuno stadio del CP, l'ARM esegue *global pooling* per calcolare un vettore di attenzione (via conv+BN+ReLU) che ri-pesa le feature e ne affina la qualità.

3.3 Architettura della Rete

BiSeNet fonde SP e CP. Lo SP è efficiente perché poco profondo; il CP sfrutta *downsampling* rapido con backbone leggero. I due percorsi elaborano in parallelo, aumentando l'efficienza.

Feature Fusion Module (FFM) → Poiché SP e CP sono a livelli semantici diversi, non basta sommare. L'FFM concatena le feature, le normalizza (BN) e calcola un vettore di pesi per selezionare e combinare efficacemente canali e informazioni.

Funzione di perdita → Oltre alla loss principale sull'uscita concatenata, si aggiungono perdite ausiliarie che supervisionano gli stadi del CP (*deep supervision*). Tutte sono Softmax loss; si pesano con un parametro di bilanciamento. Le perdite ausiliarie sono solo in training.

5. BiSeNet V2: Bilateral Network with Guided Aggregation for Real-time Semantic Segmentation

1. Introduzione

La segmentazione semantica richiede accuratezza elevata ma anche velocità. Le architetture classiche privilegiano l'accuratezza, ma non la velocità. Al contrario, le versioni ottimizzate in tempo reale riducono i dettagli spaziali, peggiorando le prestazioni.

L'osservazione centrale è che **dettagli a basso livello e semantica ad alto livello possono essere trattati separatamente**. Nasce così BiSeNet V2, che introduce due rami specializzati (Detail e Semantic) e un livello di aggregazione che li combina.

2. Concetti chiave di BiSeNet V2

- **Detail Branch:** ottimizzato per i dettagli spaziali, usa molti canali e pochi strati, mantenendo le feature ad alta risoluzione.
- **Semantic Branch:** profondo e leggero, con pochi canali e forte riduzione dimensionale. Cattura il contesto grazie al *receptive field* ampio e al pooling globale.
- **Aggregation Layer:** fonde i due rami in modo bilanciato, evitando che le informazioni spaziali o semantiche vengano dominate.

- **Booster Training:** durante l'addestramento vengono aggiunti "segmentation heads" ausiliari, che migliorano l'apprendimento ma sono eliminati in inferenza.

3. Architettura di BiSeNet V2

La rete è composta da tre parti:

1. **Detail Branch** – Struttura a tre stadi che estrae mappe ricche di dettagli spaziali. Non utilizza collegamenti residui per ridurre i costi di memoria.
2. **Semantic Branch** – Include blocchi come lo **Stem Block** (riduzione rapida della risoluzione), il **Context Embedding Block** (pooling globale per il contesto), e il **Gather-and-Expansion Layer** (per ampliare il campo recettivo).
3. **Guided Aggregation Layer** – Combina i due rami usando la semantica come guida per raffinare i dettagli spaziali.
4. **Booster Training** – Migliora ulteriormente la precisione durante l'addestramento, senza penalizzare la velocità in uso reale.

6. SegmentMelfYouCan: ABenchmark for Anomaly Segmentation

Abstract

Le reti di segmentazione semantica di ultima generazione sono addestrate su insiemi chiusi di classi. Ciò le rende inadatte a gestire oggetti mai visti prima, un problema critico per applicazioni di sicurezza come la guida autonoma. Riconoscere e localizzare tali oggetti è fondamentale, soprattutto se si trovano sulla carreggiata.

Per colmare le lacune degli attuali benchmark, che spesso si basano su dati sintetici o etichette incoerenti, viene introdotto **SegmentMelfYouCan**, un benchmark con due compiti:

- **Anomaly Segmentation:** rilevare qualsiasi categoria non vista durante l'addestramento.
- **Obstacle Segmentation:** rilevare qualsiasi oggetto sulla strada, noto o ignoto.

Il benchmark include due dataset reali e annotati, con una suite di test che combina metriche pixel-wise e component-wise. Sono state valutate diverse architetture baseline, mostrando che i nuovi dataset aumentano la difficoltà e la varietà del problema.

1. Introduzione

Dataset come Cityscapes, BDD100k o COCO hanno favorito il progresso nella segmentazione semantica. Tuttavia, i modelli addestrati su questi dati non riescono a gestire classi fuori distribuzione, risultando sovra-confidenti anche di fronte a oggetti sconosciuti.

Alcuni benchmark precedenti, come **Fishyscapes** e **CAOS**, hanno affrontato il problema, ma con limiti: uso di dati sintetici, bassa diversità, etichette incoerenti. SegmentMelfYouCan mira a superare queste barriere, introducendo dataset reali, annotati con cura e con scenari diversificati.

2. Lavori Correlati

2.2 Metodi per Anomaly e Obstacle Segmentation

- **Post-processing della classificazione:** regolazione delle confidence (softmax, ODIN, Mahalanobis).
- **Incertezza bayesiana:** dropout Monte Carlo, ensemble.
- **Addestramento con dati OoD:** uso di dataset esterni per migliorare la sensibilità alle anomalie.
- **Modelli generativi:** ricostruzione/rigenerazione delle immagini per evidenziare regioni ignote.

- **Altri domini:** produzione industriale e medicina hanno definizioni differenti di anomalia.

3. Descrizione del Benchmark

Track e Dataset

- **RoadAnomaly21:** 100 immagini reali con oggetti anomali (es. animali, veicoli sconosciuti) in qualsiasi punto della scena. Include classi “anomaly”, “not anomaly” e “void”.
- **RoadObstacle21:** 327 immagini con ostacoli sulla carreggiata (es. giocattoli, tronchi, slitte). La regione di interesse è la strada; classi: “obstacle”, “not obstacle”, “void”. Sono incluse condizioni varie di illuminazione e meteo.

Metriche

- **Pixel-level:** Area under Precision-Recall Curve (AuPRC) e False Positive Rate a 95% True Positive Rate (FPR95).
- **Component-level:** uso di metriche basate su IoU e precisione per valutare il rilevamento indipendentemente dalla dimensione degli oggetti. Questo livello è più sensibile alle piccole anomalie.

4. Esperimenti Numerici

- I metodi tradizionali di classificazione si adattano male alla segmentazione di anomalie/ostacoli.
- Le tecniche generative (resynthesis, SynBoost, inpainting) offrono risultati migliori sia a livello pixel sia componente.
- **Maximized entropy** raggiunge i punteggi migliori (fino a 86% AuPRC e F1 component-wise del 49%), evidenziando l'importanza di usare dati proxy per anomalie/ostacoli.
- Le valutazioni cross-dataset (LostAndFound, Fishyscapes) confermano che RoadAnomaly21 e RoadObstacle21 pongono sfide nuove e più complesse.

7. Enhanced Isotropy Maximization Loss: Seamless and High-Performance Out-of-Distribution Detection Simply Replacing the SoftMax Loss

1. Introduzione

Nelle applicazioni reali, oltre a classificare correttamente, un sistema deve capire se un input appartiene alle classi note (in-distribution) o è fuori distribuzione (OOD). Molti metodi OOD esistenti:

- presuppongono accesso a campioni OOD $\Rightarrow$ stime di performance ottimistiche e irrealistiche;
- usano *input preprocessing* (p.es. ODIN), che moltiplica tempi/consumi in inferenza;
- introducono **calo di accuratezza** in classificazione (anche >2% in certi casi).

2. Contesto

Metodi di addestramento/fine-tuning

- **SoftMax + Maximum Probability Score:** semplice, nessun calo di accuratezza, ma prestazioni OOD modeste.
- **Scaled Cosine:** richiede modifiche al classificatore, spesso degrada la classificazione.
- **Generalized ODIN:** evita campioni OOD per la validazione, ma introduce iperparametri e preprocessing (inefficiente in test).

- **IsoMax**: soluzione *seamless* (nessun tuning, nessun dato extra, inferenza efficiente, accuratezza intatta).

3. IsoMax+ e Minimum Distance Score

3.1 Richiamo: IsoMax (rilevazione entropica)

Si consideri un input x , le feature $f(x)$ e un prototipo p_j per ciascuna classe j . Definiamo la distanza (euclidea non al quadrato):

$$d_j = \|f(x) - p_j\|_2.$$

La **IsoMax loss** (sostituto di SoftMax) massimizza l'entropia fuori distribuzione usando una **scala entropica** E_s (costante fissata globalmente), applicata a distanze negative (equivalenti ai logit):

$$L_{\text{IsoMax}} = -\log \frac{\exp(-E_s d_k)}{\sum_j \exp(-E_s d_j)} \quad (\text{classe corretta } k).$$

In inferenza (con il *trick* di massimizzazione dell'entropia), si ottengono probabilità:

$$P(y = j | x) = \frac{\exp(-d_j)}{\sum_\ell \exp(-d_\ell)}.$$

Il **punteggio entropico** (negativa dell'entropia di P) è il detector OOD "classico" in IsoMax.

3.2 IsoMax+: isometrizzazione delle distanze

Due criticità di IsoMax: $\|f(x)\|$ e $\|p_j\|$ non normalizzati possono **biasare** l'entropia.

IsoMax+ introduce tre interventi:

1. **Normalizzazione** delle feature e dei prototipi:

$$\tilde{f} = \frac{f(x)}{\|f(x)\|_2}, \quad \tilde{p}_j = \frac{p_j}{\|p_j\|_2}.$$

2. **Parametro di scala delle distanze** $d_s > 0$ (apprendibile) per ripristinare l'escursione:

$$D_j = d_s \|\tilde{f} - \tilde{p}_j\|_2.$$

3. **Inizializzazione casuale** dei prototipi (gaussiana a varianza unitaria), coerente con la normalizzazione.

La **IsoMax+ loss** diventa:

$$L_{\text{IsoMax+}} = -\log \frac{\exp(-E_s D_k)}{\sum_j \exp(-E_s D_j)}.$$

In inferenza (rimozione di E_s):

$$P(y = j | x) = \frac{\exp(-D_j)}{\sum_\ell \exp(-D_\ell)}.$$

Intuizione: la normalizzazione rende **isometriche** le distanze per tutte le classi e tutte le feature; d_s (scalare e *class-independent*) ripristina un range informativo. Ne deriva una separazione più coerente tra ID (vicini ai prototipi) e OOD (lontani).



3.3 Minimum Distance Score (MDS)

Si propone di sostituire il punteggio entropico con il **punteggio di distanza minima**, già calcolato per la classificazione:

$$\text{MDS}(x) = \min_j D_j.$$

- **Costo zero**: è il valore che decide la classe (quella col prototipo più vicino).
- Con **IsoMax+** funziona molto bene ($\text{ID} \downarrow D_j$, $\text{OOD} \uparrow D_j$).
- Con **IsoMax** (senza isometria) il MDS è molto meno efficace.

5. Risultati e discussione

Accuratezza di classificazione

Modelli addestrati con **IsoMax+** non subiscono cali di accuratezza rispetto a SoftMax/IsoMax (talvolta migliorano, es. ResNet-110 su CIFAR-100).

Perché funziona

- **Isometria** $\Rightarrow$ l'in-distribution si addensa vicino ai prototipi; l'out-of-distribution viene spinto lontano.
- **MDS** sfrutta direttamente la geometria indotta dalla loss (nessun costo aggiuntivo), massimizzando il *gap* ID/OOD.

8. Mitigating Neural Network Overconfidence with Logit Normalization

Introduzione

Un classificatore affidabile deve classificare correttamente i campioni **in-distribution (ID)** e rifiutare (o segnalare come “sconosciuti”) gli input **out-of-distribution (OOD)**. Il metodo ingenuo basato sul massimo della softmax (MSP) è debole, perché le reti tendono a **soprastimare la confidenza** anche su input lontani dal training. Si propone **LogitNorm**, che normalizza i logit in addestramento per attenuare l'overconfidence alla radice, senza cambiare architettura

Motivazione e idea

Con la cross-entropy standard, la norma dei logit cresce durante il training, anche quando gli esempi sono già classificati correttamente. Questo non cambia l'accuratezza ma porta a **confidenze softmax molto alte**, riducendo la separazione ID/OOD. **LogitNorm** impedisce questa deriva: fissa la norma dei logit e costringe l'ottimizzazione a lavorare sulla **direzione** (coerenza con l'etichetta), non sulla **magnitudine**.

Metodo: Logit Normalization

In addestramento, i logit $f(x; \theta)$ vengono normalizzati a norma costante prima della cross-entropy:

$$\tilde{f}(x; \theta) = \frac{f(x; \theta)}{\tau \|f(x; \theta)\|_2},$$

dove $\tau > 0$ è un iperparametro di “temperatura”.

La loss di LogitNorm diventa:

$$\mathcal{L}_{\text{LogitNorm}}(x, y) = -\log \frac{\exp(\tilde{f}_y)}{\sum_{i=1}^k \exp(\tilde{f}_i)}.$$

Così la rete non può più ridurre la loss gonfiando i logit, ma solo migliorando l'allineamento direzionale con l'etichetta corretta.

Conclusioni

LogitNorm è un sostituto semplice ed efficace della cross-entropy:

- migliora sensibilmente la rilevazione OOD,
- mantiene l'accuratezza ID,
- favorisce calibrazione e confidenze realistiche,
- è facile da implementare (solo sostituire la loss).

9. ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation

I. Introduzione

Gli scenari urbani presentano alta complessità. Le CNN, nate per classificazione, sono state adattate alla segmentazione con risultati eccellenti, ma spesso con costi computazionali elevati o tempi non real-time. Le **residual network** hanno consentito reti molto profonde e accurate, ma ingombranti. In applicazioni IV contano velocità, compattezza su embedded e basso consumo. ERFNet nasce per risolvere il **trade-off** tra accuratezza ed efficienza **migliorando il blocco convolutivo**, non aggiungendo complessità ausiliaria.

II. Lavori correlati

Le prime reti “fully convolutional” trasformavano modelli di classificazione in segmentatori end-to-end, ma con uscite grossolane; varianti hanno introdotto **skip** (per fondere dettagli superficiali) o decoder con **unpooling** più fine. Altre linee hanno usato **CRF** per affinare i bordi, aumentando però i tempi.

Le architetture basate su **ResNet** hanno imposto i **blocchi residuali** per superare la degradazione all'aumentare della profondità, portando a risultati top (DeepLab, RefineNet, FRRN, LRR). Tuttavia, molte di queste soluzioni sono troppo pesanti per un uso on-board. Sul fronte opposto, **ENet** privilegia l'efficienza sacrificando precisione. ERFNet mira a un **compromesso superiore**.

III. ERFNet: Architettura proposta

A. Layer residuali fattorizzati

I blocchi residuali classici (non-bottleneck a 3×3 e bottleneck con riduzione interna) hanno pregi e limiti: il bottleneck scala meglio in costo, ma può degradare la capacità quando la rete cresce; il non-bottleneck ha buona espressività, ma più pesante.

ERFNet re-ingegnerizza il non-bottleneck introducendo il **non-bottleneck-1D**: sostituisce ogni convoluzione 2D con **coppie di convoluzioni 1D** (orizzontale e verticale) con non-linearità intermedia. Benefici:

- **Meno parametri e computazione**, già notevoli anche su kernel “piccoli”.
- **Migliore compattezza** (condivisione strutturale) e **regolarizzazione intrinseca**.
- **Capacità espressiva** comparabile al non-bottleneck, ma con il profilo di costo più vicino al bottleneck (anzi, spesso più veloce in pratica).

Questo blocco è plug-in e riusabile in altre architetture (classificazione o segmentazione), e rende più “conveniente” aumentare la **larghezza** (numero di feature map) senza costi proibitivi.

B. Design dell'architettura

ERFNet adotta un **encoder-decoder** lineare e snello:

- **Encoder:** sequenza di blocchi residuali non-bottleneck-1D con **tre downsampling** distribuiti (all'inizio e a metà), così da:
 - aumentare il campo ricettivo e il contesto,
 - contenere la computazione.In alcuni blocchi si usano **convoluzioni dilatate** per ampliare il contesto senza aumentare parametri o stride. È usato **dropout** regolare nei blocchi intermedi (più "forte" di approcci affini) per ridurre overfitting. Il **downsampler** concatena un ramo conv con stride e un ramo max-pool, risultando efficiente e informativo.
- **Decoder: compatto**, con pochi layer dedicati solo a riportare la risoluzione all'input tramite **deconvoluzioni** (transposed conv) con stride. Non si riutilizzano indici di pooling, semplificando memoria/implementazione e mantenendo pari o leggermente migliore accuratezza. **Non** si usano skip encoder-decoder lunghi (non hanno dato guadagni empirici in questa architettura).

Il risultato è un percorso sequenziale che produce directly la mappa di segmentazione a risoluzione piena.

IV. Esperimenti

B. Confronto dei layer residuali

Sostituendo, dentro un'architettura di riferimento, i blocchi bottleneck con non-bottleneck e con **non-bottleneck-1D**, emergono trend chiari:

- Aumentare la **larghezza** (più canali per blocco) incrementa molto la capacità (miglior fitting sul train).
- Il **non-bottleneck-1D** fornisce **più capacità per costo** rispetto alle alternative: il salto di parametri è contenuto e il tempo d'inferenza cresce poco.
- La versione 1D mostra **minor overfitting** a parità di capacità aggiunta, suggerendo una migliore regolarizzazione strutturale.

Queste evidenze guidano la scelta del blocco come mattone base di ERFNet.

C. Valutazione dell'architettura proposta

Su Cityscapes, ERFNet:

- raggiunge **alta accuratezza media**, con prestazioni **molto elevate** su classi dominanti (strada, edificio, vegetazione, cielo, auto),
- mantiene **buoni risultati** su classi rare o "sottili" (muri, recinzioni, semafori, segnali, pedoni, ciclisti, moto, bus, treni), spesso critiche per la IoU,
- mostra **generalizzazione solida**: scarto ridotto tra validazione e test, a conferma della robustezza.
- la variante **pretrained** è migliore in media (come noto in letteratura), ma la versione **from-scratch** resta competitiva: utile quando non si possono usare dati esterni.

D. Confronto con lo stato dell'arte

- **Vs metodi più accurati** (RefineNet, FRRN, DeepLabv2, LRR): ERFNet ottiene **accuratezza comparabile**, ma con **architettura molto più leggera** e **inferenza sensibilmente più rapida**; le reti concorrenti usano pipeline multiple, ResNet molto profondi, o raffinamenti costosi.
- **Vs metodi più veloci** (ENet, SQ): ERFNet è **quasi altrettanto veloce**, ma **decisamente più accurato**, soprattutto nelle classi meno frequenti o con geometrie complesse.

F. Qualità della segmentazione (qualitativa)

Le uscite di ERFNet risultano:

- **nitide e coerenti** sulle classi principali,
- **più precise** di reti focalizzate solo sulla velocità, nelle aree lontane e negli oggetti sottili (pedoni, segnali, pali),

- con **pochi errori di confusione** tra classi semantiche vicine (es. bus vs truck), fisiologici data la scarsa presenza di esempi e la forma complessa.

10. Scaling Out-of-Distribution Detection for Real-World Settings

1. Introduzione

L'OOD detection consente a sistemi ML di adottare politiche conservative o deferire a un umano, ed è parte centrale della sicurezza dei modelli. I lavori recenti usano reti profonde, ma per lo più su dataset piccoli. Nei contesti su larga scala (ImageNet, Places) emergono sfide nuove (classi molto fini). Gli autori mostrano che il classico massimo della softmax (MSP) non scala bene e propongono MaxLogit, che risulta superiore in multiclass su larga scala, multilabel e segmentazione. Per valutazioni realistiche e senza leakage, introducono Species (OOD per modelli pre-addestrati su ImageNet-21K) e CAOS (anomaly segmentation realistico con StreetHazards simulato e BDD-Anomaly reale). Contrariamente ad affermazioni recenti, i Vision Transformer pre-addestrati su ImageNet-21K non risolvono “da soli” l'OOD detection in questi scenari.

2. Lavori correlati

Multiclass OOD. Dalla baseline MSP si è passati a metodi con outlier exposure, GAN per near-OOD, tarature e post-hoc. Molte valutazioni però sono indulgenti (tuning su anomalie simili a quelle di test). Questo lavoro evita ogni sovrapposizione e tuning sulle classi OOD di test e punta a dataset grandi con molte classi.

Anomaly segmentation. Alcuni benchmark trattano l'intera immagine come OOD o usano “copia-incolla” di oggetti (con artefatti di bordo/illuminazione che rendono il compito artificiale). Lost & Found è reale ma con poche categorie e anomalie molto piccole; Fishyscapes soffre di incongruenze contestuali. Qui si privilegia inserimento fisicamente coerente in simulazione e immagini reali varie e moderne.

3. OOD su classificazione multiclass su larga scala

Limite di MSP. Con molte classi simili, la softmax distribuisce la probabilità su più classi affini, abbassando la confidenza anche su esempi in-distribuzione: MSP penalizza ingiustamente immagini ID “fine-grained”.

Proposta MaxLogit. Usare il massimo logit non normalizzato come punteggio OOD evita la dipendenza dal numero di classi e dalla redistribuzione di probabilità, risultando più robusto nei grandi vocabolari.

Benchmark Species. Nuovo dataset OOD con oltre 700k immagini di specie disgiunte dalle classi di ImageNet-21K. Consente valutazioni senza sovrapposizioni tra pretraining e test.

Setup e risultati (in sintesi). Modelli addestrati su ImageNet-21K (versione pulita con split) sono valutati su Species e su scenari in-distribuzione come ImageNet-1K/Places con vari OOD. MaxLogit supera sistematicamente MSP e altre baseline su tutti i modelli (ResNet, ViT, MLP-Mixer). I risultati mostrano che il semplice “scalare” i ViT non rende triviale l'OOD detection quando si eliminano le sovrapposizioni di classi.

4. OOD su classificazione multilabel

Motivazione. Nelle scene reali, le immagini contengono più etichette, e MSP non si applica naturalmente (richiede softmax).

Risultato chiave. MaxLogit generalizza bene al multilabel e risulta il migliore su tutte le metriche e dataset, offrendo una baseline semplice ed efficace anche in questo setting.

5. Benchmark CAOS per anomaly segmentation

StreetHazards (simulato). Anomalie 3D inserite in ambienti di guida con Unreal Engine/CARLA, garantendo coerenza di illuminazione/orientamento e varietà (oggetti, condizioni meteo, ore del giorno). Train/val/test in città diverse, 12 classi ID più una classe “anomaly” solo a test.

BDD-Anomaly (reale). Ricavato da BDD100K: si rimuovono moto, treno, bicicletta dal train/val e si usano come anomalie nel test. Più grande e vario di Lost & Found; anomalie più estese in media.

Metodi valutati.

- MSP per pixel;
- probabilità di “background” come proxy OOD;
- incertezza con dropout (varianza posteriore per pixel);
- autoencoder con loss di ricostruzione;
- ramo di confidenza (porting del confidence branch);
- MaxLogit applicato per pixel.

Risultati (in breve). MaxLogit è il migliore su StreetHazards e BDD-Anomaly. Il “background” come detector funziona male; l’autoencoder soffre la complessità delle scene. AUPR è generalmente basso (forte sbilanciamento), ma c’è margine: MaxLogit migliora nettamente e riduce i falsi positivi tipici dei confini semantici rispetto a MSP.